# Assignment 02 Report

## Course

CSE323: Advanced Embedded Systems Design
CSE411/CSE411s: Real-Time and Embedded Systems Design

## Assignment

Assignment 02: Wrong Stack Pointer Restoration Demo

## Objective

The objective of this assignment is to demonstrate what happens when a task context is restored using the wrong stack pointer. Instead of restoring the correct saved exception frame, the program intentionally shifts the stack pointer by one word for Task 2. This causes the decoded program counter and status register fields to become invalid, which is then detected by the corruption check.

## System Design

The program builds two synthetic Cortex-M exception frames, one for `Task1()` and one for `Task2()`, using `BuildInitialFrame()`. A simulated context-restore routine named `AttemptContextRestore()` is called periodically from `SysTick_Handler()`.

The restore routine selects the next task, chooses the correct saved stack pointer in `g_restoredSp`, intentionally shifts it for Task 2 using `g_faultyRestoreSp = g_restoredSp + 1`, decodes the frame, and checks whether the decoded `pc` and `xpsr` are valid.

## Variables Used for Verification

The main watch variables used during debugging were `g_ui32SysTickCount`, `g_restoreAttemptCount`, `g_currentTaskId`, `g_nextTaskId`, `g_contextCorruptionDetected`,

`g_expectedPc`, `g_observedPc`, `g_faultSignature`, `g_currentSp`, `g_restoredSp`, `g_faultyRestoreSp`, `g_lastDecodedFrame.pc`, and `g_lastDecodedFrame.xpsr`.

# Debug Method

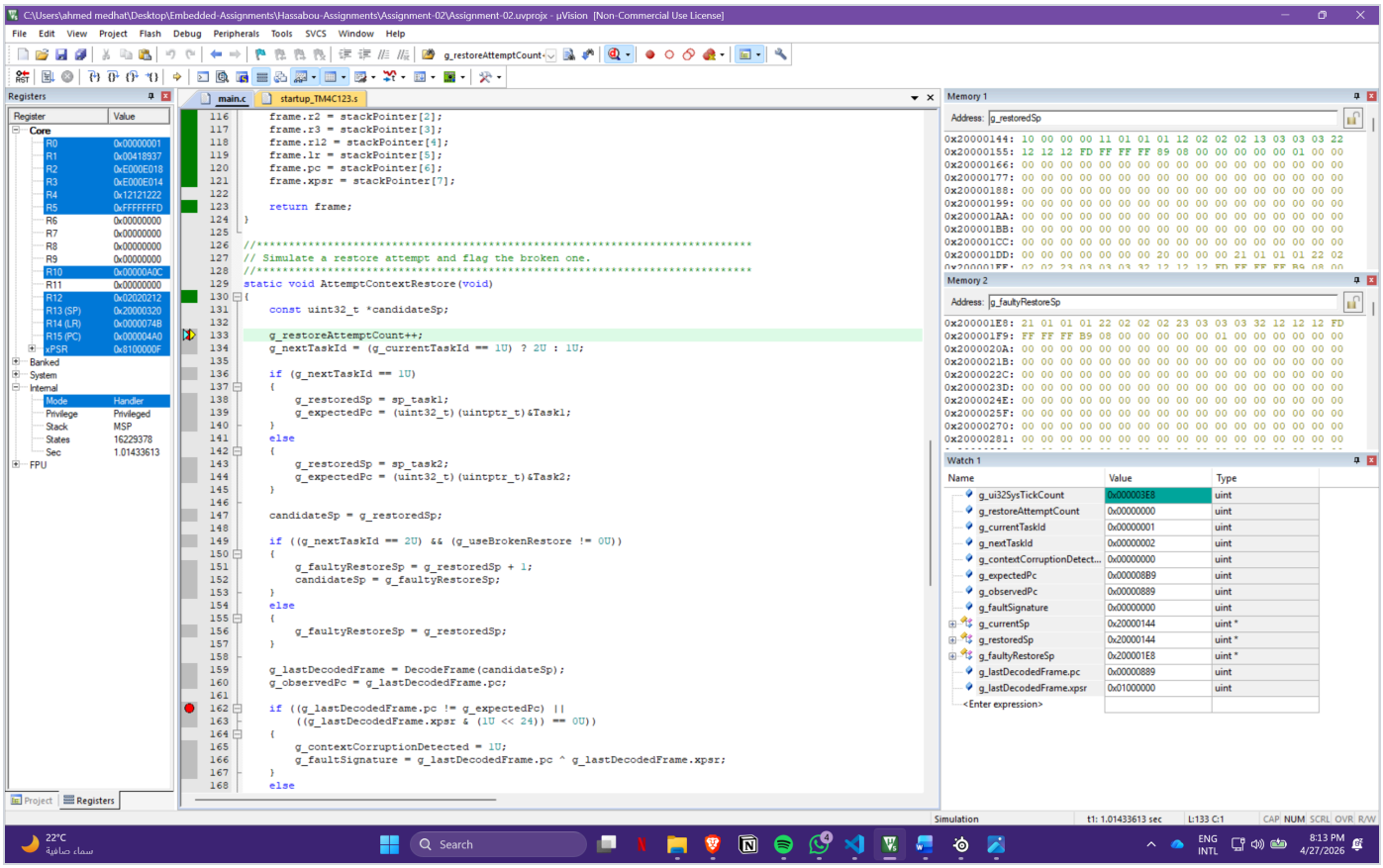
Two breakpoints were used inside `AttemptContextRestore()`: one at the start of the restore attempt and one at the corruption-check `if` condition. The Memory windows were used to compare `g_restoredSp` and `g_faultyRestoreSp`.

# Screenshot Verification and Analysis

## Screenshot 1: Correct restore pointer versus faulty shifted pointer

File: `Screenshot 2026-04-27 201316.png`

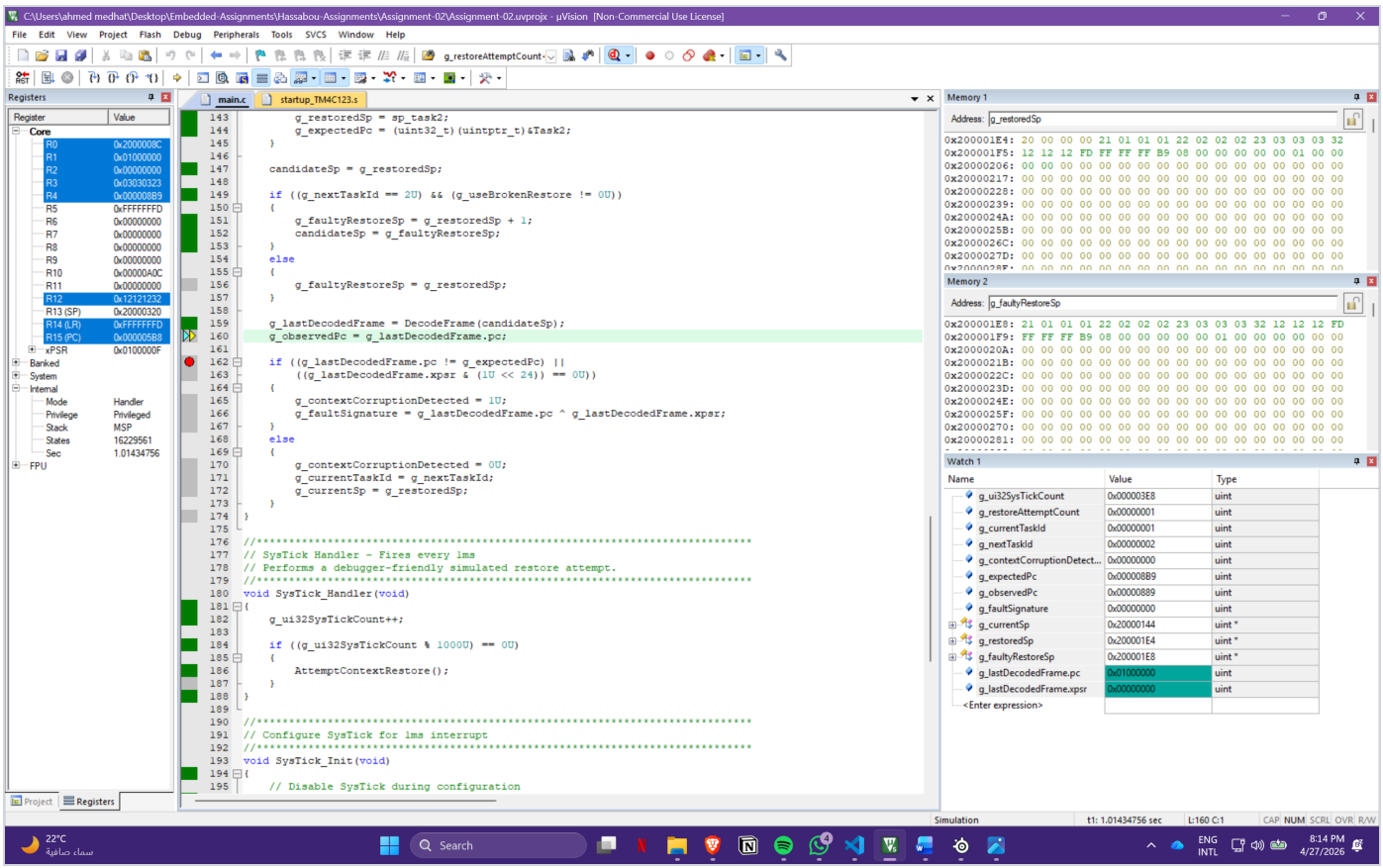
This screenshot shows the first restore attempt after the SysTick counter reaches `0x000003E8`. The next task selected is Task 2. The watch values show `g_restoredSp = 0x200001E4` and `g_faultyRestoreSp = 0x200001E8`, proving that the faulty restore pointer is shifted by exactly one word.



## Screenshot 2: Corrupted frame decode

File: `Screenshot 2026-04-27 201426.png`

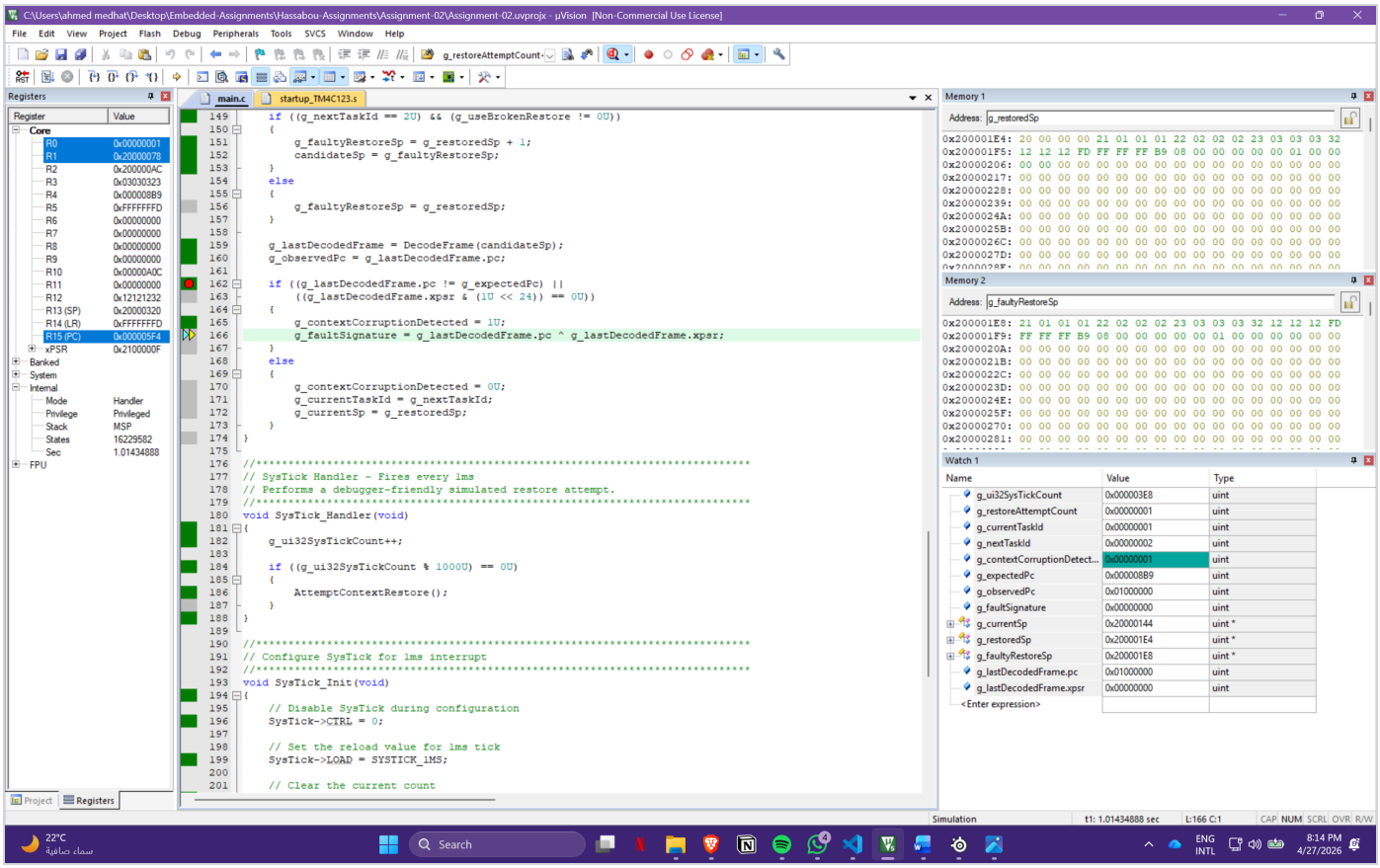
This screenshot shows the state immediately after decoding the frame from the faulty pointer. The expected PC for Task 2 is `0x000008B9`, but the observed decoded values are corrupted: `g_observedPc = 0x01000000`, `g_lastDecodedFrame.pc = 0x01000000`, and `g_lastDecodedFrame.xpsr = 0x00000000`.




## Screenshot 3: Corruption detected and fault signature generated

File: `Screenshot 2026-04-27 201451.png`

This screenshot shows the program after entering the corruption-handling path. The watch values show `g_contextCorruptionDetected = 1`, `g_expectedPc = 0x000008B9`, `g_observedPc = 0x01000000`, and `g_faultSignature = 0x01000000`.




## Experimental Result Summary

The debugger results show a complete cause-and-effect chain: the correct saved frame exists at `g_restoredSp`, the restore address is intentionally changed to `g_faultyRestoreSp = g_restoredSp + 1`, decoding from that shifted address corrupts the frame fields, the decoded `pc` no longer matches `g_expectedPc`, the decoded `xpsr` becomes invalid, and the corruption flag is raised.

## Conclusion

The assignment objective was achieved successfully. The implementation demonstrates that restoring a task context from the wrong stack pointer causes corrupted exception-frame decoding. The debugger evidence clearly shows the pointer shift, the resulting mismatch between expected and observed program counter values, and the final corruption detection flag.